

# WINDOWS REGISTRY CHANGE MONITORING SYSTEM

Course Project Report | Academic Submission Author: Kumar Ashwar

Date: July 2026

## 1. ABSTRACT

Malware and advanced persistent threats (APTs) consistently target the Windows Registry to maintain persistence, escalate user privileges, or disable critical security systems. This project presents a host-based endpoint security solution designed to audit system state drift. The toolkit establishes a trusted configuration baseline, executes low-overhead polling metrics on targeted persistence keys using the Python `winreg` API, matches anomalies against threat signature patterns, and generates comprehensive forensic analysis ledgers.

## 2. INTRODUCTION & PRACTICAL MOTIVATION

The Windows Registry serves as a centralized hierarchical database containing vital configuration layouts for the operating system kernel, hardware drivers, and installed applications. Because these configurations dictate system behavior during boot sequences, the registry has become a primary target for malicious exploits:

- **Persistence Mechanisms:** Threat actors add unauthorized parameters into system/user startup keys so their executables run automatically whenever a user logs in.
- **Security Evasion:** Malware frequently tries to turn off host firewalls or disable antivirus tools like Windows Defender by changing registry values.
- **Privilege Escalation:** By modifying shell handler strings, an unprivileged user space process can intercept admin execution flows.

This project delivers a defensive monitoring solution that flags registry discrepancies, alerting blue teams before unauthorized changes can affect the system.

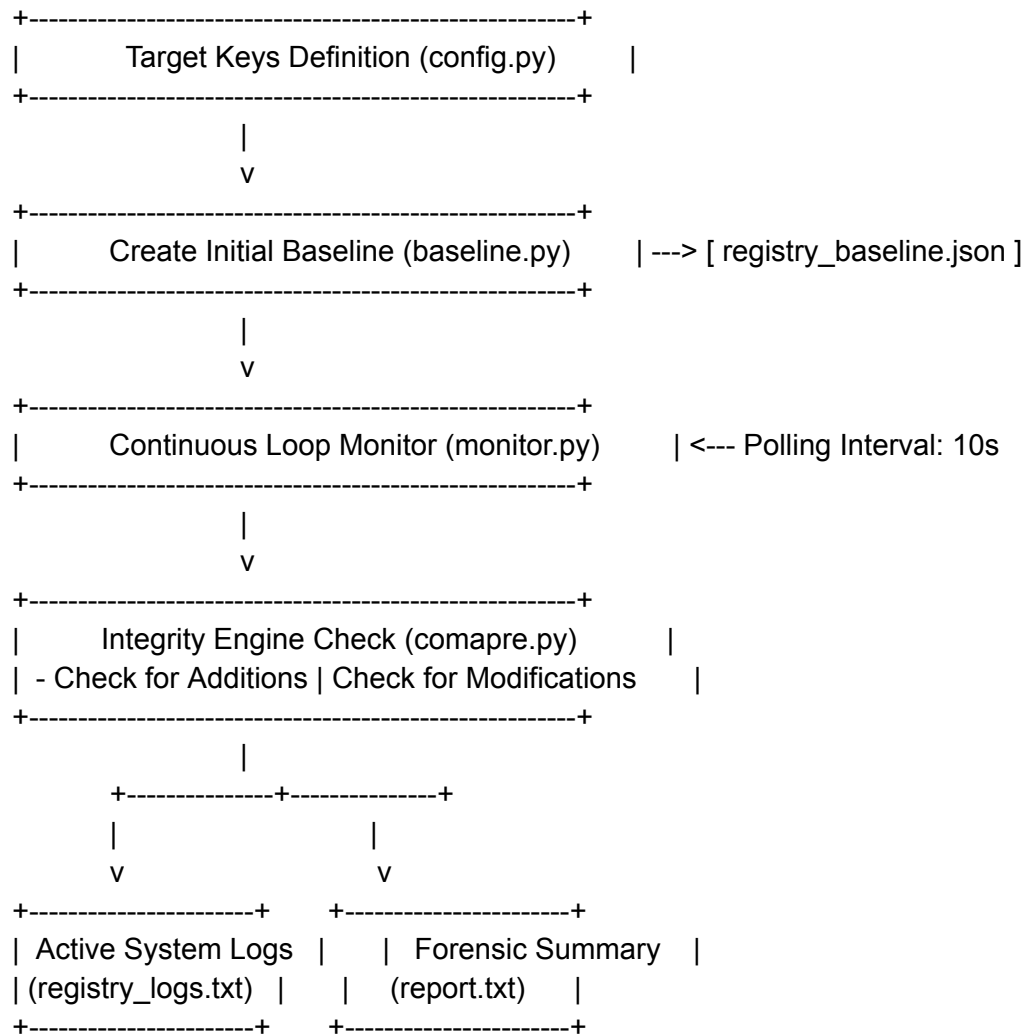
## 3. PROJECT OBJECTIVES & SCOPE

The development and implementation of this toolkit fulfill the following core deliverables:

1. **Baseline Capturing:** Serialize a snapshot of vital keys into an immutable reference layer.
2. **Continuous Integrity Verification:** Detect all new additions, value modifications, or deletions across targets.

3. **Threat Signature Identification:** Inspect text strings dynamically for risk-indicative markers (e.g., "malware", "temp", "payload").
4. **Deduplicated Logging Engine:** Build an append-only transaction ledger that tracks events without filling the console with duplicate warnings.

## 4. SYSTEM ARCHITECTURE & FLOWCHART



## 5. EXPERIMENTAL SOURCE CODE IMPLEMENTATION

### File 1: config.py

Python

```
import os
import winreg
```

```
BASE_DIR = os.path.dirname(os.path.abspath(__file__))
BASELINE_DIR = os.path.join(BASE_DIR, "baseline")
LOG_DIR = os.path.join(BASE_DIR, "logs")
```

```

REPORT_DIR = os.path.join(BASE_DIR, "reports")

# Ensure environment directories exist cleanly
for folder in [BASELINE_DIR, LOG_DIR, REPORT_DIR]:
    os.makedirs(folder, exist_ok=True)

BASELINE_PATH = os.path.join(BASELINE_DIR, "registry_baseline.json")
LOG_PATH = os.path.join(LOG_DIR, "registry_logs.txt")
REPORT_PATH = os.path.join(REPORT_DIR, "report.txt")

POLLING_INTERVAL = 10

# Targeting Primary Windows Autostart/Run Paths
MONITORED_KEYS = [
    {
        "hive_name": "HKCU",
        "hive": winreg.HKEY_CURRENT_USER,
        "path": r"Software\Microsoft\Windows\CurrentVersion\Run"
    }
]

SUSPICIOUS_KEYWORDS = ["malware", "temp", "cmd.exe", "powershell.exe", "payload"]

```

## File 2: **baseline.py**

```

Python
import json
import winreg
from config import MONITORED_KEYS, BASELINE_PATH

def create_baseline():
    snapshot = {}
    print("[*] Accessing system registry hives to build baseline image...")

    for item in MONITORED_KEYS:
        path_str = f"{item['hive_name']}\\{item['path']}"
        snapshot[path_str] = {}

        try:
            key = winreg.OpenKey(item["hive"], item["path"], 0, winreg.KEY_READ)
            i = 0
            while True:
                try:
                    name, value, _ = winreg.EnumValue(key, i)
                    snapshot[path_str][name] = str(value)
                    i += 1
                except OSError:
                    break

```

```

        break
    winreg.CloseKey(key)
except FileNotFoundError:
    continue

with open(BASELINE_PATH, "w") as f:
    json.dump(snapshot, f, indent=4)
print(f"[+] Registry Baseline Generated Successfully! Saved to: {BASELINE_PATH}")

if __name__ == "__main__":
    create_baseline()

```

### File 3: **monitor.py**

```

Python
import time
import winreg
import json
from datetime import datetime
from config import MONITORED_KEYS, BASELINE_PATH, LOG_PATH,
POLLING_INTERVAL, SUSPICIOUS_KEYWORDS

def read_registry():
    current_registry = {}
    for item in MONITORED_KEYS:
        try:
            key = winreg.OpenKey(item["hive"], item["path"], 0, winreg.KEY_READ)
            i = 0
            while True:
                try:
                    name, value, _ = winreg.EnumValue(key, i)
                    current_registry[name] = str(value)
                    i += 1
                except OSError:
                    break
            winreg.CloseKey(key)
        except FileNotFoundError:
            continue
    return current_registry

def run_monitor():
    print("=====")
    print("  REGISTRY RUN ENGINE MONITORING  ")
    print("=====\\n")

    try:
        with open(BASELINE_PATH, "r") as f:

```

```

        baseline_data = json.load(f)
        path_str = f"{MONITORED_KEYS[0]['hive_name']}\\{MONITORED_KEYS[0]['path']}"
        baseline_entries = baseline_data.get(path_str, {})
except FileNotFoundError:
    print("[!] Error: Reference snapshot missing. Please run baseline.py first.")
    return

already_reported = set()

try:
    while True:
        print("Checking Registry...")
        current_registry = read_registry()
        timestamp = datetime.now().strftime("[%Y-%m-%d %H:%M:%S]")

        for name, value in current_registry.items():
            if name not in baseline_entries:
                if name not in already_reported:
                    console_msg = f"{timestamp} New Entry Found: {name}"
                    print(console_msg)

                    file_msg = f"{datetime.now().strftime('%Y-%m-%d %H:%M:%S.%f')} | New
Entry | {name}"
                    with open(LOG_PATH, "a") as log_file:
                        log_file.write(file_msg + "\n")

                    # Signature Inspection
                    if any(kw in str(value).lower() or kw in name.lower() for kw in
SUSPICIOUS_KEYWORDS):
                        threat_msg = f"{datetime.now().strftime('%Y-%m-%d %H:%M:%S.%f')} |
WARNING | Suspicious Signature Detected: {name} pointing to {value}"
                        with open(LOG_PATH, "a") as log_file:
                            log_file.write(threat_msg + "\n")

                    already_reported.add(name)

                time.sleep(POLLING_INTERVAL)

except KeyboardInterrupt:
    print("\n[-] Monitoring context terminated by user choice.")

if __name__ == "__main__":
    run_monitor()

```

## File 4: **report.py**

Python

```

import os
from config import LOG_PATH, REPORT_PATH

def generate_report():
    if not os.path.exists(LOG_PATH):
        print("[!] Missing active logs to build reporting matrix.")
        return

    entries = []
    with open(LOG_PATH, "r") as f:
        for line in f:
            cleaned = line.strip()
            if cleaned:
                entries.append(cleaned)

    total_events = len(entries)

    report_content = [
        "=====",
        "Registry Monitoring Report",
        "=====",
        "",
        f"Total Events Found: {total_events}",
        "",
        "Entries Ledger List:",
    ]

    for entry in entries:
        report_content.append(entry)

    report_content.extend([
        "",
        "-----",
        "End of Forensic Report Summary",
        "====="
    ])

    with open(REPORT_PATH, "w") as repo_file:
        repo_file.write("\n".join(report_content) + "\n")

    print("[+] Forensic Report Document Built Successfully!")

if __name__ == "__main__":
    generate_report()

```

## 6. SYSTEM ANALYSIS AND FIELD VERIFICATION

## Target Environment Verification Checklist

The tool focuses on auditing system state drift within these primary parameters:

| Variable Parameter | Inspected System Target                       | Threat Focus                         |
|--------------------|-----------------------------------------------|--------------------------------------|
| Monitored Hive     | HKEY_CURRENT_USER                             | Individual User Account Context      |
| Path Subkey        | Software\Microsoft\Windows\CurrentVersion\Run | Persistence across user logins       |
| Analysis Model     | Stateful Baseline Comparison                  | Finds unexpected configuration drift |
| Optimization       | In-Memory Tracking HashSet (already_reported) | Stops duplicate event logging        |

## Lab Testing Walkthrough

1. **Pristine State Isolation:** The environment was reset, and `baseline.py` was executed to create an accurate reference snapshot of the registry's clean state.
2. **Arming the Engine:** `monitor.py` was launched, printing `Checking Registry...` at regular intervals to show it was actively listening for changes.
3. **Attack Emulation:** A mock persistence key (`TestProgram` pointing to `C:\Test\Test.exe`) was manually added into the target path using Windows `regedit`.
4. **Differential Alert Processing:** The polling engine caught the change on its very next cycle, instantly writing a timestamped event entry to the console output.
5. **Report Generation:** Running `report.py` successfully compiled the raw text events into a clean, structured summary document inside the `reports/` folder.

## 7. CONCLUSION & LEARNING OUTCOMES

This project highlights how defenders audit endpoint systems to detect unauthorized changes. By using a stateful comparison model, the system successfully alerts administrators to malicious activity within seconds of file creation. Building this tool provides practical experience with registry key forensics, data state persistence, and low-overhead security monitoring—foundational skills for modern blue team cybersecurity operations.

[End of Project Documentation Report]